SAE 15 – Traitement de données

Projet n°7

R&T 1ère année

Objectif :

Le but est de lire un emploi du temps, de nettoyer les données, de calculer les heures de cours, et de les trier par matière et par type (Cours Magistral, Travaux Dirigés, Travaux Pratiques) pour un étudiant de première année (BUT1).

7. Projet n°7

Aider le service de communication à promouvoir le BUT auprès des lycéens

Pour mieux expliquer aux lycéens, le service communication doit connaitre la répartition horaire de chaque matière.

Objectif

Afficher, pour le BUT1, la répartition (en volume horaire) des CM, TD, TP de chaque matière.

Cahier des charges fonctionnel

- Les données seront extraites d'ADE.
- Le script prend en compte toute l'année universitaire, c'est à dire du 1 septembre au 31 août.

Fonctions spécifiques au projet

Le script doit être découpé en différentes fonctions, pour suivre les principes de structuration du code.

Code :

```
import numpy as np
import pandas as pd

# Chargement du fichier CSV
df = pd.read_csv("calendrier.csv", sep=";")
df.columns = df.columns.str.strip()

def calculer_duree(h_debut, h_fin):
    debut = pd.to_datetime(h_debut, format="%H:%M:%S", errors='coerce')
    fin = pd.to_datetime(h_fin, format="%H:%M:%S", errors='coerce')
    if pd.isnull(debut) or pd.isnull(fin):
        return 0
```

```
    return (fin - debut).total_seconds() / 3600

def extraire_type(row):
    groupe = str(row.get("Groupe", "")).upper()

    if "COURS" in groupe:
        return "CM"
    elif "TD" in groupe or "TDA" in groupe or "TDB" in groupe:
        return "TD"
    elif "TP" in groupe or "PT" in groupe:
        return "TP"
    else:
        return "Autre"

def extraire_matiere(summary):
    return str(summary).split(";")[0].strip().split()[0]

def repartition_BUT1():
    # On garde uniquement les lignes du BUT1
    df_but1 = df[df["Description"].str.contains("1A", case=False, na=False)].copy()

    # Calcul de la durée brute
    df_but1["Duree"] = df_but1.apply(
        lambda row: calculer_duree(row["HStart"], row["HEnd"]),
        axis=1
    )

    # Extraction du type CM / TD / TP
    df_but1["Type"] = df_but1.apply(extraire_type, axis=1)

    # Extraction de la matière
    df_but1["Matiere"] = df_but1["Summary"].apply(extraire_matiere)

    # Application des coefficients selon le type
    def ajuster_volume(row):
        if row["Type"] == "CM":
            return row["Duree"]              # CM commun
        elif row["Type"] == "TD":
            return row["Duree"] / 2          # 2 groupes TD
        elif row["Type"] == "TP":
            return row["Duree"] / 4          # 4 groupes TP
        else:
            return 0

    df_but1["Duree_corrigee"] = df_but1.apply(ajuster_volume, axis=1)

    # Tableau croisé final
    tableau = df_but1.pivot_table(
        values="Duree_corrigee",
        index="Matiere",
        columns="Type",
```

```
        aggfunc=np.sum,
        fill_value=0
    )

    return tableau

def afficher_repartition(tableau):
    for matiere in tableau.index:
        cm = tableau.at[matiere, "CM"] if "CM" in tableau.columns else 0
        td = tableau.at[matiere, "TD"] if "TD" in tableau.columns else 0
        tp = tableau.at[matiere, "TP"] if "TP" in tableau.columns else 0
        print(f"{matiere} : CM={cm:.2f} h, TD={td:.2f} h, TP={tp:.2f} h \n")

tableau = repartition_BUT1()
afficher_repartition(tableau)
```

Explication du code :

Ce script suit une logique classique de Data Science : ETL (Extract, Transform, Load).

1. Nettoyage et préparation (`read_csv`, `strip`)

Le fichier CSV issu d'un logiciel d'emploi du temps (comme ADE ou Celcat) est souvent "sale".

- Les colonnes peuvent avoir des espaces cachés (ex: `" Groupe "`). La commande `df.columns.str.strip()` règle cela immédiatement. Sans ça, appeler `df["Groupe"]` provoquerait une erreur.

2. Manipulation des Dates (`pd.to_datetime`)

Pandas ne devine pas automatiquement que "08:30:00" est une heure. On doit le forcer avec `to_datetime`.

- L'astuce mathématique : Soustraire deux objets temps donne un `Timedelta`. La méthode `.total_seconds()` est le moyen le plus sûr d'obtenir un nombre pur, qu'on divise par 3600 pour avoir des heures décimales (ex: 1h30 devient 1.5h, ce qui est calculable, alors que "1:30" ne l'est pas).

3. Logique Métier (`extraire_type`, `ajuster_volume`)

C'est ici que réside l'intelligence du script adaptée à l'université.

- Le problème : Dans un fichier CSV d'emploi du temps, si un prof fait cours au Groupe A de 8h à 10h et au Groupe B de 10h à 12h, le fichier contient 2 lignes.
- Si on fait la somme simple, on trouve 4 heures. Or, un étudiant n'a eu que 2 heures de cours.
- La solution du script :
    - Il détecte si c'est un TD ou un TP.
    - Il part du principe qu'il y a 2 groupes de TD (donc il divise le total par 2) et 4 groupes de TP (donc il divise par 4).

- Cela permet d'estimer "La charge horaire pour un étudiant moyen" à partir de "La charge horaire totale des enseignants".

4. Agrégation (`pivot_table`)

C'est l'équivalent du Tableau Croisé Dynamique sur Excel.

- Il transforme une liste de milliers de lignes (chaque créneau de cours) en un tableau synthétique compact.
- Il regroupe tout ce qui a le même nom de matière (`index="Matiere"`).
- Il sépare les heures dans des colonnes distinctes (`columns="Type"`).

En Résumé de ce que fait le script :

Il prend un fichier CSV brut et illisible d'emploi du temps complet, et il te sort un rapport propre qui dit : " Pour la matière R1.01, un étudiant a eu 10h de CM, 12h de TD et 4h de TP ".

Résultats :

Voici une petite partie du résultat lors de l'exécution du code où l'on peut voir la matière tout à gauche et son volume d'heures par rapport au Cours Magistraux (CM), Travaux Dirigés (TD) et Travaux Pratiques (TP).

```
SAE1.01 : CM=2.00 h, TD=12.00 h, TP=5.00 h

SAE1.02 : CM=4.00 h, TD=18.00 h, TP=7.50 h

SAE1.03 : CM=0.00 h, TD=12.00 h, TP=12.00 h

SAE1.04 : CM=2.00 h, TD=12.00 h, TP=9.00 h
```